

DocAlign Architecture Diagram Blueprints

Owner: Martha Wood (Documentation), with source material from Priya Anand (Engineering)

Status: Draft for review

Date: July 2026

Source: Derived from the DocAlign 1.0 engineering handoff

Purpose

This document specifies the three architecture diagrams called for in the DocAlign 1.0 documentation: their content, structure, labels, and the design conventions they share. It serves two functions: as a build sheet for producing final, polished versions in a diagramming tool (e.g. [Security-first diagramming for teams | draw.io](#), Lucidchart, or Mermaid), and as a record of the design decisions behind each diagram so they can be maintained consistently.

The three diagrams are:

- **Figure 1 — Four-component architecture** (what DocAlign is made of)
 - **Figure 2 — DocAlign's position in a CI/CD pipeline** (where DocAlign sits in the flow)
 - **Figure 3 — The Accept / Reject / Revise workflow** (the human-in-the-loop decision flow)
-

Shared design conventions

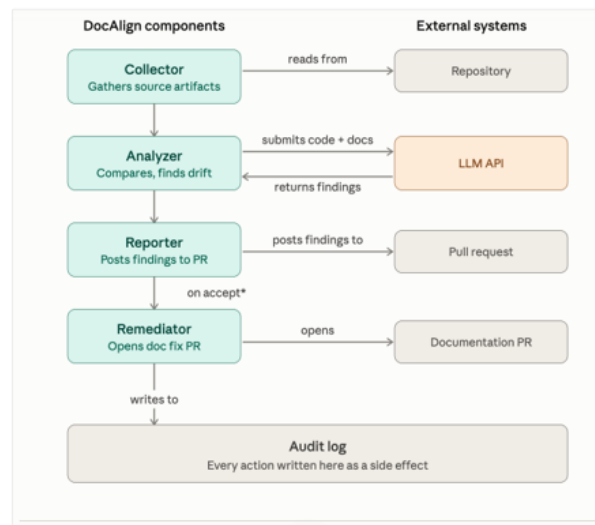
These conventions apply across all three diagrams and should be preserved in the final production versions. They are also candidates for the DocAlign Style Guide.

- **Figures have titles and numbers.** Every figure carries a number and a descriptive title, and is referred to by number in cross-references.
- **Actors are named explicitly when they vary.** In process diagrams where different agents perform different steps, each step names its actor (e.g. "CI/CD pipeline builds the code," "DocAlign checks for drift," "Writer confirms the diff"). This shows who does what rather than implying a single subject.
- **Labels use parallel grammar.** Within a diagram, box labels follow a consistent grammatical structure (e.g. subject-verb-object).
- **Connection labels use the most precise verb that is accurate.** Arrow labels name the action taking place, choosing the most technical term that is genuinely correct for the mechanism

(e.g. "reads from" for file reading rather than "queries," which would imply an API call). Accuracy governs the choice, not how technical the word sounds.

- **Color encodes category, not decoration.** DocAlign's own components share one color; external systems share another; human-in-the-loop checkpoints use a distinct color (amber) consistently across diagrams; success/outcome and failure states use their own colors.
- **Arrows are treated consistently.** Every connection terminates in an arrowhead, regardless of the arrow's length.
- **The audit log is rendered consistently** across diagrams as a full-width element receiving actions as a side effect.

Figure 1 — Four-component architecture



Purpose. Shows DocAlign's internal structure: its four components, the order in which they operate, and the external systems each connects to.

Components (left column, in sequence, shown in DocAlign's component color):

1. **Collector** — Gathers source artifacts
2. **Analyzer** — Compares, finds drift
3. **Reporter** — Posts findings to PR
4. **Remediator** — Opens documentation fix PR

The four are connected top to bottom by unlabeled arrows indicating the processing sequence. (The sequence itself conveys data flow, so these arrows are not labeled.)

External systems (right column) and their labeled connections:

- Collector — **reads from** — Repository

- Analyzer — **submits code + docs** to — LLM API; LLM API — **returns findings** to — Analyzer (two separate labeled arrows showing the round trip)
- Reporter — **posts findings to** — Pull request
- Remediator — **opens** — Documentation PR

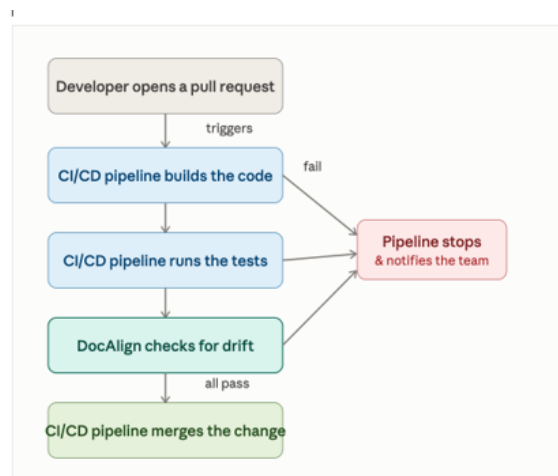
Conditional connection:

- Reporter to Remediator is labeled "**on accept***" with a footnote: *The Remediator runs only when a writer accepts an auto-remediable finding. For accept / reject / revise details, see Figure 3, "The Accept/Reject/Revise workflow."*

Audit log: A full-width element at the bottom, labeled "Every action written here as a side effect," with the component column connecting into it ("writes to").

Open item: Confirm the Collector's mechanism with Engineering — whether it reads files from the checked-out repository ("reads from") or queries a remote API ("queries"). The handoff's description of running as a GitHub Action / Jenkins step suggests file reading, so "reads from" is used pending confirmation.

Figure 2 — DocAlign's position in a CI/CD pipeline



Purpose. Shows where DocAlign sits within the larger CI/CD flow, from a developer's pull request through to merge, as one stage among the pipeline's existing stages.

Main flow (top to bottom), each box naming its actor in parallel subject-verb-object form:

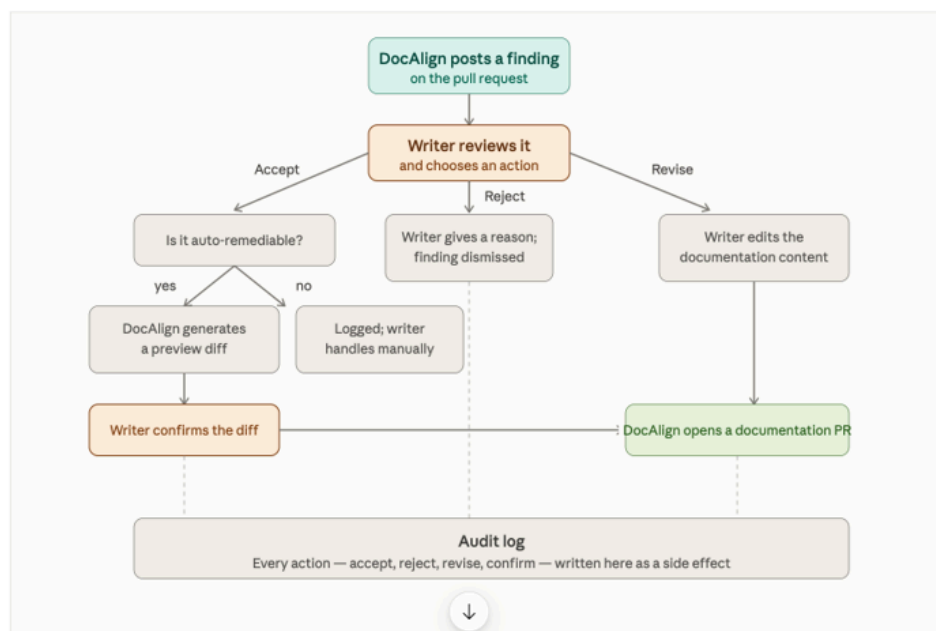
1. **Developer opens a pull request** (neutral/trigger color) — labeled connection "triggers" to the next stage
2. **CI/CD pipeline builds the code** (existing-stage color)
3. **CI/CD pipeline runs the tests** (existing-stage color)

4. **DocAlign checks for drift** (DocAlign color, visually emphasized as the subject of the diagram)
— connection labeled "all pass" to merge
5. **CI/CD pipeline merges the change** (success color)

Failure path: A single "Pipeline stops & notifies the team" box (failure color) off to the side, with "fail" arrows from the build, test, and DocAlign stages, showing that any stage failing stops the pipeline.

Notes. "Build" and "test" are representative existing stages, included to establish that DocAlign slots in among other stages; real pipelines vary. The actor "CI/CD pipeline" is used (rather than naming GitHub Actions or Jenkins specifically) because DocAlign supports both integrations, so a generic actor better reflects that it works with either. The label "DocAlign checks for drift" uses the settled term "drift."

Figure 3 — The Accept / Reject / Revise workflow



Purpose. Shows the human-in-the-loop decision flow after DocAlign posts a finding: the writer's three choices and their consequences, including the confirmation gate on auto-remediation. Human-decision points use the amber human-in-the-loop color.

Entry (top):

1. **DocAlign posts a finding** on the pull request (DocAlign color)
2. **Writer reviews it and chooses an action** (amber — human checkpoint)

Three branches from the decision, with their first-level outcome boxes aligned in a row:

- **Accept** → decision box "Is it auto-remediable?" which branches:
 - **yes** → "DocAlign generates a preview diff" → **Writer confirms the diff** (amber — second human checkpoint) → documentation PR
 - **no** → "Logged; writer handles manually"
- **Reject** → "Writer gives a reason; finding dismissed" (terminates here; no PR produced)
- **Revise** → "Writer edits the documentation content" → documentation PR

Shared outcome: "DocAlign opens a documentation PR" (success color), reached by both the confirmed-Accept path and the Revise path.

Audit log: A full-width element at the bottom, labeled "Every action — accept, reject, revise, confirm — written here as a side effect," with the action points connecting into it.

Design notes. The three branches share an aligned first-level row of outcome boxes, then descend to different depths reflecting their genuinely different downstream consequences (Accept has the most; Reject terminates immediately). The amber color marks the two human-in-the-loop checkpoints — the initial choice and the diff confirmation — reinforcing that autonomy yields to human judgment at each consequential point. This diagram is the cross-reference target from Figure 1.

Production notes

These blueprints were designed as draft renderings for review. Final production versions should be built in a diagramming tool for clean, exportable, editable output ([Security-first diagramming for teams | draw.io](#) / [Flowchart Maker & Online Diagram Software](#) is free and widely used; Mermaid suits a docs-as-code workflow by generating diagrams from text; Lucidchart is a common commercial option). When produced, assign each figure its number and title, apply the shared design conventions above, and update the Figure 1 cross-reference to point to the final Figure 3.